

# Cloud handoff — #486, the "tips unknown" admission is folded away and counted as a ref

**Written:** 2026-08-25 · **By:** max (CLI session on Tom's box) · **For:** a cloud Claude Code session on `tom2025b/git-vista`

**The smallest of the six, and the one where the previous session did most of your work for you.** The defect is measured, the fix is named, and there is already a `# [should_panic]` test in the tree whose assertions are correct and waiting. Read it before you write anything of your own.

```
task_id: gv-486-tips-unknown-fold
issue: 486
milestone: – # a #329 successor
repo: tom2025b/git-vista
base: main
branch: fix/486-tips-unknown-not-folded
sign_commits_as:
  name: Claude_Max
  email: 262510778+tom2025b@users.noreply.github.com
sign_artifacts_as: max
adr_number: 0081 # ASSIGNED. Do not pick "the next free" c
allowed_paths:
  - crates/git-vista-core/src/activity.rs
  - crates/git-vista-server/src/planner/fetch.rs # only if the st
  - docs/adr/
forbidden_paths:
  - design-docs/
  - ci/browser/**
  - crates/git-vista/src/**
  - handoff.md
merge_order: NOT independent – shares crates/git-vista-core/src/act
```

---

## The defect, already measured

---

`journal_unobserved` (`planner/fetch.rs:542`) fires when `git fetch` **succeeded** and only the re-read of `refs/remotes/<remote>/*` failed. It journals one entry with `ref_name: None` and both oids `Obs::Unknown`:

*"fetched from 'origin', but refs/remotes/origin could not be re-read afterwards, so which remote-tracking refs moved is unknown: ..."*

If the fetch succeeded it very likely moved refs — and git wrote a reflog line for every one. The admission carries no `new_oid`, so it matches no reflog line in `assemble_feed`'s attribution step and **suppresses none of them**. They all survive, the admission joins them, and `fold_ref_update_bursts` folds the lot together:

```
journal:  1 × "fetched from 'origin' ... (tips unknown – git could not  
reflog:   4 × "fetch origin: fast-forward"  
feed:     1 × "fetch – 5 refs updated"
```

**Two harms in one line of output.** The deliberate admission is replaced by a confident count — the exact distinction `Obs::Unknown` rather than `Obs::Absent` exists to preserve, destroyed in the direction of false confidence. And the count is wrong anyway: four refs moved, and the admission is itself counted as the fifth.

## What is already in the tree, and must not be re-derived

---

`crates/git-vista-core/src/activity.rs` carries all of this, written by the session that found it:

- **The corrected doc comment**, lines ~605-620. The comment *used* to claim the entry was safe because it is journaled "instead of per-ref entries, never alongside them, so it is always a run of one". That reasoning accounts for the journal and forgets git's reflog — **the same shape of mistake that got the first #329 attempt reverted in `0a7ba777`**. The comment now says so.
- **A `#[should_panic]` test at ~line 1284**, documented as *F2*, whose doc comment names the fix outright:

**Fixing this:** exclude it from folding. `ref_name: None` with both oids `None` is the shape `journal_unobserved` alone produces. Then delete the `#[should_panic]` — the assertions below are already right.

**Read that test's assertions before writing any of your own, and do not rewrite them to fit your implementation.** They were written by someone who had measured the failure. If your fix cannot satisfy them as they stand, your fix is the thing to change.

Full evidence: `docs/investigations/2026-08-25-issue-329-fetch-feed-volume.md`.

---

## The one thing to be careful about

---

The named fix — exclude `ref_name: None` with both oids `None` from folding — is almost certainly right. **Check it is not too wide before you ship it.** Is `journal_unobserved` genuinely the only producer of that shape? If some other path also emits an entry with no ref name and no oids, excluding the shape excludes those too, and a burst that *should* fold stops folding.

Grep for every construction site of that shape rather than trusting the comment — that trust is precisely what produced the defect you are fixing. If the shape turns out to be shared, say so and propose a narrower discriminator (an explicit marker on the entry, for instance) in **ADR 0081** rather than widening the exclusion quietly.

The comment's original claim, by the way, **is** true in exactly one case: a fetch that moved nothing at all, which is the one case with no reflog lines. A fix that keeps folding correct there as well is the complete one.

---

## What you cannot run

---

**The browser leg does not run in a cloud container** — the kernel reports `landlock_abi=-1` and INV-13 gives the server no degraded tier; installing `bwrap` changes nothing. This is a pure-core change and should not need it. If you find yourself wanting a browser assertion, the change has reached the frontend, which is outside `allowed_paths` — stop and say so in the PR.

`cargo test -p git-vista-core` and `cargo test --workspace` are yours and must be green. Two tests in `git-vista-server` flake under parallel execution because they race on the process-global current repository (#438):

`recovery_center::tests::a_stale_claimed_undo_is_refused_and_the_branch_is_left_alone` and

`state::tests::selection_flow_carries_mode_and_gates_writes`. Re-run before believing either is your doing.

---

## Acceptance

---

1. Four refs moved with the re-read failing renders as **the admission, intact** — not "5 refs updated", and not "4 refs updated" either. The user must see that something could not be read.
2. The `#[should_panic]` on the F2 test is **deleted** and the test passes on its own assertions, unedited.
3. A fetch that moved nothing at all still folds as it did before. Add the test if one does not exist — the doc comment says this is the case the original claim was true for, so it is the case a too-wide fix breaks.
4. Every test pinning this is proved able to go red **two different ways** — remove the exclusion, and weaken it (exclude on `ref_name: None` alone, say). One `caught` verdict is not proof; a

Git-Vista test survived one mutation and caught another on 2026-08-22, and either alone gives the wrong verdict. Note: no production path currently constructs a Fetch/Pull event with `ref_name: None` and a `Some oid`, or `ref_name: Some` with both oids `None` — so on the F2 test's existing fixture, "exclude on `ref_name: None` alone" and the full fix are behaviorally identical, and that mutation will not go red on its own. Add a synthetic fixture event exercising that combination (it need not come through a real code path) so the second mutation has something to catch.

5. **ADR 0081** records the discriminator chosen and why — especially if the grep above found the shape is shared.

`docs/adr/README.md` index updated.

5. `cargo fmt --all`, `cargo clippy --all-targets -- -D warnings`, `cargo test --workspace` green.

7. PR body says `Closes #486`. **Never delete the branch.**

---

**Signed:** max · 2026-08-25T07:24:00-04:00